

Estructura Jerárquica de un Proyecto en MicroPython

Este documento describe la organización de archivos estándar para el desarrollo de aplicaciones en **MicroPython**. A diferencia de los entornos de desarrollo web tradicionales, las placas de desarrollo (como ESP32, Raspberry Pi Pico o STM32) tienen recursos limitados, por lo que la estructura debe ser limpia y eficiente.

Vista General del Proyecto

Así es como debe quedar la estructura de archivos en un proyecto típico. Se recomienda que estos elementos y la estructura correspondiente (jerarquía entre archivos, directorios, documentación, etcétera) lo guardes en carpetas específicas en tu computadora. Cada carpeta corresponderá a una aplicación o Laboratorio del curso. Cuando se requiera ejercitar en el sistema uC un Laboratorio o experimento, en Thonny buscaremos la carpeta que contiene la estructura del proyecto (como la mostrada abajo), conectamos el dispositivo uC y trasladamos a él los archivos con los que trabajaremos. Al concluir satisfactoriamente el ensayo de una aplicación en un sistema uC se sugiere que se copien los archivos resultantes hacia la PC, como resguardo y para documentación.

```
mi-proyecto-micropython/  
├── src/  
│   ├── boot.py  
│   ├── main.py  
│   ├── config.py  
│   └── lib/  
│       ├── mi_libreria.py  
│       └── ssd1306.py  
├── tests/  
│   └── test_main.py  
├── tools/  
│   └── upload.sh  
├── README.md  
└── requirements.txt
```

Descripción de los Componentes

1. Archivos Raíz del Sistema (**src/**)

Estos son los archivos que se **suben directamente** a la memoria interna de la placa (directorio raíz del microcontrolador).

- **boot.py**: Es el **primer archivo que se ejecuta** al encender o reiniciar la placa. Se utiliza estrictamente para configuraciones críticas del sistema que solo deben ocurrir una vez.
 - *Ejemplo de uso*: Configurar la conexión Wi-Fi; inicializar el sistema de archivos o definir pines de depuración; establecer las rutas de trayectoria, (*PATH*), de recursos; entre otras cosas).

- **main.py**: Es el archivo principal de la aplicación. **Se ejecuta automáticamente justo después** de que **boot.py** finaliza con éxito. Aquí reside el bucle principal (**while True**) de tu aplicación.
- **config.py**: Archivo centralizado para almacenar constantes, credenciales de red (SSID/Password), tokens de API y configuraciones de pines (GPIO). Separarlo ayuda a no modificar el código base al cambiar de entorno. En este archivo se define la *personalidad* del sistema, cómo se distribuyen sus recursos (terminales o pines) conectados a los periféricos y recursos.

2. Directorio de Librerías (**src/lib/**)

MicroPython busca automáticamente módulos en esta carpeta cuando usas la sentencia **import**.

- **lib/**: Carpeta donde se almacenan todos los controladores de hardware externos (sensores, pantallas) y tus propios módulos personalizados.
 - *Módulos de terceros*: Controladores como **ssd1306.py** (para pantallas OLED) o **bme280.py** (sensores ambientales).
 - *Módulos propios*: Código de sensores, actuadores o procesos que son reutilizables, estos se construyen para que sean estructurados en clases o funciones para su mejor utilización.

3. Entorno de Desarrollo Local (No se suben a la placa)

Archivos necesarios para la gestión del proyecto en tu computadora, pero que **no** deben ocupar espacio en el almacenamiento limitado del microcontrolador. Son auxiliares para el desarrollo, prueba, experimentación y documentación del proyecto.

- **tests/**: Scripts de prueba (unit testing) para validar la lógica de tu código antes de transferirlo al hardware.
- **tools/**: Scripts de automatización en Bash o Python para flashear el firmware, subir archivos mediante herramientas como **ampy**, **mpremote** o **rshell**, o limpiar la memoria de la placa. En nuestro curso estos elementos serán empleados también en herramientas como **Thonny** y **Visual Studio Code**.
- **requirements.txt**: Lista de paquetes de MicroPython (**micropython-lib**) necesarios para el desarrollo, útiles si usas gestores como **mip** o **pipkin**. Son para ampliar la funcionalidad del desarrollo de aplicaciones.
- **README.md**: Documentación del proyecto que explica cómo instalar el firmware, las conexiones de los pines (pinout) y cómo poner en marcha la aplicación. Obligatorio. Aquí se encuentra lo necesario para conocer cómo está conectado cada pin o terminal del uC a sensores, actuadores, o periféricos.

Buenas Prácticas

1. **Mantén **boot.py** lo más corto posible**: Si ocurre un error fatal en este archivo, puedes perder el acceso a la placa por puerto serial (REPL).
2. **Usa archivos **.mpy** (Precompilados)**: Para proyectos grandes con poca memoria RAM, compila tus archivos **.py** a **.mpy** usando la herramienta **mpy-cross** antes de subirlos. Esto ahorra memoria de ejecución y espacio de almacenamiento. Lo harán en partes avanzadas del curso.
3. **Evita subcarpetas profundas**: El sistema de archivos de MicroPython (usualmente LittleFS o FAT) funciona mejor y de manera más rápida con estructuras de directorios planas.

```
=====
==== SFHM@2026
```

=====

=====